TextPulse Code Walkthrough

Prepared for Shubham · 31 October 2026

1. The map

TextPulse has two phases that never run at the same time: **training**, which you run once and which learns from 1.6 million messages, and **serving**, which runs continuously and uses what was learned. Every file belongs to one of these phases, or to setup and testing.

```
textpulse/
├── requirements.txt        SETUP    libraries to install
├── .gitignore              SETUP    what Git should not upload
├── data/                   SETUP    the 3 downloaded datasets
├── src/
│   ├── config.py           SHARED   every path and setting
│   ├── clean_text.py       SHARED   cleans one message
│   ├── load_data.py        TRAIN    reads the CSV files
│   ├── train.py            TRAIN    learns and saves the models
│   └── analyzer.py         SERVE    predicts + explains one message
├── scripts/
│   ├── download_data.py    SETUP    fetches the datasets
│   └── benchmark.py        TRAIN    scores all 1.6M records for speed
├── models/                 OUTPUT   what training creates
├── app/
│   ├── main.py             SERVE    web server + live stream engine
│   └── static/             SERVE    the dashboard (HTML, CSS, JS)
├── tests/                  CHECK    automated tests
└── docs/                   GUIDES   this kind of explanation
```

The order things run in:

1. `pip install -r requirements.txt` installs the libraries.
2. `python -m src.train` runs `train.py`, which calls `load_data.py` to read the CSVs, uses `clean_text.py` inside TF-IDF, and writes everything into `models/`.
3. `python -m scripts.benchmark` (optional) loads the saved models through `analyzer.py` and scores every record.
4. `python -m uvicorn app.main:app` starts `main.py`, which loads the models through `analyzer.py` and starts the stream.
5. Your browser opens `http://127.0.0.1:8000`, receives `index.html`, `style.css` and `app.js`, and `app.js` connects back to `main.py` to receive live results.

One idea connects everything: **`models/` is the hand-off point.** Training writes files into it; the server only reads them. That's why you train once and then start the dashboard as often as you like without retraining.

A note on `python -m src.train`: the `-m` means "run this as a module", using dots instead of slashes. It runs from the project folder, so imports like `from src.config import ...` work. Running `python src/train.py` directly would fail with `No module named 'src'`.

2. Setup files

requirements.txt: the shopping list of libraries

```
scikit-learn>=1.3      machine learning: TF-IDF, Naive Bayes, Logistic Regression, metrics
pandas>=2.0            reading CSV files into tables
numpy>=1.24            fast number arrays (used inside pandas and scikit-learn)
joblib>=1.3            saving trained models to files and loading them back
fastapi>=0.110         the web framework (like ASP.NET Core)
uvicorn[standard]>=0.27  the web server that runs FastAPI (like Kestrel)
kagglehub>=0.2         downloading datasets from Kaggle
pytest>=7.0            running the automated tests
httpx>=0.25            used by FastAPI's testing tools
```

`>=1.3` means "version 1.3 or newer". `pip install -r requirements.txt` reads this file and installs every line. It's the Python equivalent of NuGet's package list in a `.csproj` file.

.gitignore: what Git must not upload

```
data/*              datasets: 290 MB, and owned by their publishers
!data/README.md     ...except this one file (! means "but keep")
models/*.joblib     trained models: anyone can rebuild them with src.train
models/stream_*.csv test messages: 33 MB, rebuilt by training
__pycache__/        Python's compiled cache files
.venv/              your environment: hundreds of MB, rebuilt from requirements.txt
kaggle.json         a secret token: never upload passwords or keys
```

The rule of thumb: commit what you **wrote**, ignore what can be **rebuilt** or **downloaded**, and never commit **secrets**.

data/README.md

Tells anyone who clones the project which three files to put in `data/`, where to get them, and how to cite them. Datasets have authors, and citing them is expected in academic work like your IIIT presentation.

3. src/config.py: every setting in one place

This file contains no logic, only names and values. Every other file imports from it. It's the equivalent of `appsettings.json` in ASP.NET.

```
PROJECT_DIR = Path(__file__).resolve().parent.parent
```

Read it right to left:

Piece	Value on your laptop
`__file__`	`D:\AI ML\Claude\textpulse\src\config.py` (this file)
`.resolve()`	the same path, made absolute and tidy
`.parent`	`D:\AI ML\Claude\textpulse\src`
`.parent` **again**	`D:\AI ML\Claude\textpulse`, the project folder

Because every path is built from this one starting point, the project works wherever you put it: your D: drive, my Linux machine, or a panel member's Mac.

```
DATA_DIR = PROJECT_DIR / "data"
SENTIMENT_FILE = DATA_DIR / "training.1600000.processed.noemoticon.csv"
```

With `pathlib`, the `/` operator **joins paths**. It's not division. It also picks the right separator for the operating system: `\` on Windows, `/` on Linux.

The settings, grouped:

Setting	Value	Used by	What changing it does
`TEST_SIZE`	0.2	train.py	Share of data hidden for testing (20%)
`RANDOM_SEED`	42	train.py, main.py	Same seed, same shuffle, same results every run
`VOCAB_SAMPLE_SIZE`	300,000	train.py	Messages used to build the word list. Lower it if training runs out of memory
`MAX_WORDS`	200,000	train.py	Most words and word pairs kept. Smaller means a smaller, faster, slightly less accurate model
`MIN_WORD_COUNT`	3	train.py	Ignore words seen in fewer than 3 messages (mostly typos)
`STREAM_START_SPEED`	20	main.py	Messages per second when the server starts
`STREAM_MAX_SPEED`	200	main.py	Highest speed the API accepts
`STREAM_TEXT_LIMIT`	1,000	train.py	Characters kept per message in the stream files
`RECENT_EVENTS_KEPT`	40	main.py	Feed rows a newly opened dashboard gets immediately
`SAFE_DISPLAY`	True	main.py	Hide explicit messages in the feed
`EXPLICIT_WORDS` / `EXPLICIT_WORD_STARTS`	lists	main.py	Which words trigger hiding

Names in CAPITAL letters are a Python convention meaning "a constant: set once, don't change while running".

4. src/clean_text.py: making messages consistent

A computer compares text letter by letter, so "FREE", "Free" and "free!!!" are three different words to it. This file turns every message into one consistent form before the model sees it. It runs about 3.3 million times during training: once per message for each of the 1.6 million tweets, plus the emails and SMS.

The patterns, defined once at the top:

```
LINK = re.compile(r"https?://\S+|www\.\S+")
USERNAME = re.compile(r"@\w+")
HTML_TAG = re.compile(r"<[^>]+>")
NOT_A_LETTER = re.compile(r"[^a-z' ]+")
EXTRA_SPACES = re.compile(r"\s+")
```

Pattern	Read it as	Matches
`https?://\S+`	“http”, an optional “s”, “://”, then one or more non-space characters	`http://win.biz`, `https://x.co/abc`
`|www\.\S+`	OR “www.” followed by non-spaces (`\.` is a literal dot)	`www.deals.com`
`@\w+`	“@” then one or more letters, digits or underscores	`@john`, `@team_lead`
`<[^>]+>`	“<”, then anything that isn’t “>”, then “>”	` `, `<p class="x">`
`[^a-z' ]+`	one or more characters that are NOT a–z, an apostrophe or a space (`^` inside `[ ]` means NOT)	`!!`, `123`, `:(`, `£`
`\s+`	one or more spaces, tabs or newlines	`"  "`

`re.compile` turns each pattern into a ready-to-use object once, instead of re-reading the pattern 3.3 million times.

The function, line by line, with a real example:

```
def clean_text(text):
    text = str(text).lower()             # 1
    text = LINK.sub(" url ", text)       # 2
    text = USERNAME.sub(" user ", text)  # 3
    text = HTML_TAG.sub(" ", text)       # 4
    text = NOT_A_LETTER.sub(" ", text)   # 5
    text = EXTRA_SPACES.sub(" ", text)   # 6
    return text.strip()                  # 7
```

Input: `WINNER!! Claim your FREE prize at http://win.biz now @john`

Line	After the line
1 `str(...).lower()`	`winner!! claim your free prize at http://win.biz now @john`
2 links → `url`	`winner!! claim your free prize at  url  now @john`
3 @names → `user`	`winner!! claim your free prize at  url  now  user`
4 HTML tags removed	(no tags here, unchanged)
5 non-letters → space	`winner   claim your free prize at  url  now  user`
6 many spaces → one	`winner claim your free prize at url now user`
7 `.strip()`	`winner claim your free prize at url now user`

`str(text)` on line 1 protects against a cell in the CSV that isn’t text, such as an empty value or a number.

Three design decisions worth explaining to a panel:

- **Links become the word** `url` **instead of being deleted.** The exact link changes every time, so it’s useless to learn. But *having* a link is a strong spam signal, and keeping it as a word preserves that. Your spam model gives `url` a weight of +2.35 toward spam.
- **Apostrophes are kept**, so `don't` stays one word instead of becoming `don` and `t`.
- **Small words like** `not` **are kept**, unlike the Day 3 notebook, which removed stopwords. For sentiment that matters: `I don't like this phone, not good :(` becomes `i don't like this phone not good`, and the model’s strongest clue is the pair `not good`. Removing “not” would have left “good”.

5. src/load_data.py: three Kaggle files into one table

Each dataset arrives in a different shape. This file converts all of them into one standard table with three columns, so the rest of the project never needs to know where the data came from:

text	label	source
Go until jurong point, crazy..	ham	SMS
christmas tree farm pictures …	ham	Email
@switchfoot Awww, that’s a bummer…	negative	Tweet

check_file_exists(path)

```
if not path.exists():
    raise FileNotFoundError(f"\nMissing dataset: {path.name}\nRun  python scripts/download_data.py ...")
```

Fails early with a helpful message instead of a confusing pandas error. `train.py` catches this error and skips that task, which is why training still worked before you downloaded Sentiment140.

load_sms_spam(): 5,572 SMS

```
sms = pd.read_csv(SMS_SPAM_FILE, encoding="latin-1", usecols=[0, 1])
sms.columns = ["label", "text"]
sms["source"] = "SMS"
```

- `encoding="latin-1"`: the file was saved in an older text format, not UTF-8. Without this you'd get `UnicodeDecodeError`.
- `usecols=[0, 1]`: the file has five columns, `v1, v2` plus three almost-empty ones. Keep only the first two.
- `sms.columns = [...]`: rename `v1, v2` to the standard names.
- `sms["source"] = "SMS"`: a new column with the same value on every row. The dashboard shows it in the Source column.

load_enron_spam(): 33,716 emails

```
enron["text"] = enron["Subject"].fillna("") + " " + enron["Message"].fillna("")
```

Joins each email's subject and body, because a subject like "You WON!!!" is evidence too. `fillna("")` replaces missing values with empty text first; adding text to a missing value would cause an error. This adds whole columns at once (vectorised, as on Day 2), with no loop over 33,716 rows.

load_spam_data(): the combined spam table

```
combined = pd.concat([load_sms_spam(), load_enron_spam()], ignore_index=True)
combined = combined.drop_duplicates(subset="text").reset_index(drop=True)
```

- `pd.concat` stacks the two tables on top of each other: 5,572 + 33,716 = 39,288 rows. `ignore_index=True` renumbers the rows 0 to 39,287.
- `drop_duplicates(subset="text")` removed **3,625** messages with identical text, leaving **35,663**. Spammers resend the same message, and if one copy lands in training and another in testing, the model has already seen the test question. The score would look better than it really is. This is **data leakage**, the same problem as Titanic's `alive` column on Day 2.

load_sentiment_data(): 1,600,000 tweets

```
column_names = ["target", "id", "date", "query", "user", "text"]
tweets = pd.read_csv(SENTIMENT_FILE, encoding="latin-1", header=None,
                     names=column_names, usecols=["target", "text"])
tweets["label"] = tweets["target"].map({0: "negative", 4: "positive"})
tweets = tweets.dropna(subset=["label"])
```

- `header=None, names=...`: the file has no header row, so we name the six columns ourselves.
- `usecols=["target", "text"]`: read only two of the six columns, which roughly halves the memory needed for 1.6 million rows.
- `.map({0: "negative", 4: "positive"})`: the dataset uses numbers, 0 for negative and 4 for positive. `map` swaps each number for the word using the dictionary. Any value not in the dictionary becomes missing (NaN).
- `dropna(subset=["label"])` removes those rows. There are none in this file, but the line protects against a surprise in a different copy of the data.

6. src/train.py: learning from the data

Run with `python -m src.train`. It has four functions. `main()` calls `train_task()` once for spam and once for sentiment, and `train_task()` uses `make_vectorizer()` and `score_model()`.

main(): the controller

```
tasks = [
    ("spam", load_spam_data, "spam", SPAM_MODEL_FILE, SPAM_STREAM_FILE),
    ("sentiment", load_sentiment_data, "positive", SENTIMENT_MODEL_FILE, SENTIMENT_STREAM_FILE),
]
```

Each task is a tuple of five things: a name, **the function to load its data** (passed without calling it, so no `()`), the label that counts as "positive" for scoring, and two file paths to save to.

```
results = json.loads(METRICS_FILE.read_text()) if METRICS_FILE.exists() else {}
for task_name, load_function, positive_label, model_file, stream_file in tasks:
    try:
        data = load_function()
    except FileNotFoundError as error:
        print(f"... skipped{error}"); continue
    results[task_name] = train_task(...)
METRICS_FILE.write_text(json.dumps(results, indent=2))
```

It loads earlier results first, so training only spam doesn't erase the sentiment scores. `load_function()` calls whichever loader the tuple holds. If a dataset is missing, it prints why and moves on (`continue`). At the end, all scores are saved as `models/metrics.json`, which the dashboard's model card reads.

train_task(): the six steps for one task

Step 1: split.

```
train_data, test_data = train_test_split(
    data, test_size=0.2, random_state=42, stratify=data["label"])
```

	Spam	Sentiment
All messages	35,663	1,600,000
train_data (80%)	28,530	1,280,000
test_data (20%), never seen while learning	7,133	320,000

`stratify` keeps the same label mix in both parts: the spam share is about 43% in each. `random_state=42` makes the random shuffle identical every run, which is why your laptop got almost the same scores as mine.

Step 2: learn the vocabulary.

```
vocab_sample = train_data["text"].sample(min(len(train_data), 300_000), random_state=42)
vectorizer.fit(vocab_sample)
```

`fit` reads messages and decides **which words exist**. For sentiment it reads a random 300,000 of the 1,280,000 training tweets. Collecting every word pair from all of them would need many gigabytes of memory, and a large sample finds nearly the same words. It kept **169,738** words and word pairs for sentiment and **200,000** for spam. It uses training data only; looking at test data here would be leakage.

Step 3: turn text into numbers.

```
X_train = vectorizer.transform(train_data["text"])
X_test = vectorizer.transform(test_data["text"])
```

Each message becomes a row of 200,000 numbers, one per vocabulary entry, almost all zero. The spam example `WINNER!! Claim your FREE prize at http://win.biz now` has only **11 non-zero values out of 200,000**. Python stores only those 11 in a **sparse matrix**. That's how 1.28 million tweets fit in memory. On a 2-core machine this step took 37 seconds for the tweets.

What the numbers mean: TF-IDF. These are set up in `make_vectorizer()`:

```
TfidfVectorizer(
    preprocessor=clean_text,         # run clean_text() on every message first
    token_pattern=r"[a-z][a-z']+",   # a word = a letter followed by 1+ letters or '
    ngram_range=(1, 2),              # single words AND pairs of neighbouring words
    min_df=3,                        # skip words seen in fewer than 3 messages
    max_features=200_000,            # keep the 200,000 most frequent
    sublinear_tf=True,               # count repeats as 1 + log(count), not count
    dtype=np.float32,                # half the memory of the default
)
```

- **TF (term frequency)** rewards a word that appears in this message. `sublinear_tf` stops a word repeated ten times from counting ten times as much.
- **IDF (inverse document frequency)** shrinks words that appear in almost every message, like "at" and "the". In the example, `at url` (rare) gets **0.545** while `at` (everywhere) gets only **0.117**.
- Each row is then scaled so the message's numbers have a total length of 1, so a long email doesn't automatically outweigh a short SMS.

`ngram_range=(1, 2)` is why the model understands "not good": it sees the words `not` and `good` and also the pair `not good` as its own feature.

Step 4: train two algorithms on the same data.

```
candidates = {
    "Naive Bayes": MultinomialNB(alpha=0.1),
    "Logistic Regression": LogisticRegression(C=4.0, max_iter=1000, solver="liblinear"),
}
for name, model in candidates.items():
    model.fit(X_train, train_data["label"])
    scores = score_model(model, X_test, test_data["label"], positive_label)
```

- **Naive Bayes** is Day 3's spam filter: count how often each word appears in each class, with smoothing. `alpha=0.1` is the smoothing amount (Day 3 used 1).
- **Logistic Regression** learns one **weight** per word by trial and improvement (gradient descent, Day 6). A positive weight pushes toward spam or positive; a negative weight pushes toward ham or negative. `C=4.0` controls how freely weights can grow: bigger C means less restraint. `max_iter=1000` caps the number of improvement rounds.

`.fit(X, y)` means "learn from these numbers (X) and these answers (y)". Every scikit-learn model uses the same `fit` / `predict` pattern, the same shape as the `LinearModel` class you saw on Day 1.

	Naive Bayes	Logistic Regression
Spam accuracy / training time	97.52% / 0.1 s	**97.95%** / 1 s
Sentiment accuracy / training time	80.33% / 3 s	**82.30%** / 58 s

Step 5: score on the test data, in `score_model()`:

Score	Question it answers (spam example)	Spam	Sentiment
Accuracy	Of all messages, how many were right?	97.95%	82.30%
Precision	Of messages we called spam, how many really were?	98.08%	81.66%
Recall	Of real spam, how much did we catch?	97.11%	83.31%
F1	One number balancing precision and recall	97.59%	82.47%

The **confusion matrix** behind the spam scores, from 7,133 test messages:

	Predicted ham	Predicted spam
Really ham	4,027 correct	58 good emails wrongly sent to spam
Really spam	88 spam that got through	2,960 correct

Accuracy = (4,027 + 2,960) / 7,133 = 97.95%. Precision = 2,960 / (2,960 + 58) = 98.08%. Recall = 2,960 / (2,960 + 88) = 97.11%.

```
best = max(comparison, key=lambda result: result["f1"])
```

Picks the result with the highest F1, using the `max(..., key=lambda ...)` pattern from Day 1. Then the winner is also scored separately per source: email 98.4% and SMS 95.1%.

Step 6: save.

```
pipeline = Pipeline([("tfidf", vectorizer), ("model", best_model)])
joblib.dump(pipeline, model_file, compress=3)
stream.to_csv(stream_file, index=False)
```

A `Pipeline` glues the vectorizer and the model together, so later you pass in raw text and get a prediction out in one step. `joblib.dump` saves it as a file (`spam_model.joblib`, 3.9 MB). The test messages are saved as `stream_*.csv` for the live dashboard, so the stream only ever shows messages the model never learned from.

7. src/analyzer.py: predicting and explaining one message

This file is used by the server, the benchmark and the tests. It loads a saved model and answers two questions about one message: **what is it**, and **which words made the model think so**.

word_weights(model): one number per word

```
def word_weights(model):
    if hasattr(model, "feature_log_prob_"):        # Naive Bayes
        return model.feature_log_prob_[1] - model.feature_log_prob_[0]
    return model.coef_[0]                           # Logistic Regression
```

`hasattr` checks whether the model has that attribute, which tells us which algorithm won. Either way, the result is an array of 200,000 numbers, one per vocabulary entry. Positive pushes toward class 1 (spam / positive); negative pushes toward class 0 (ham / negative). For Naive Bayes it's exactly Day 3's log-likelihood ratio, log P(word|spam) − log P(word|ham). Both models won with Logistic Regression, so they use `coef_`: the weights it learned.

`__init__`: loading once

```
class TextAnalyzer:
    def __init__(self, model_file):
        self.pipeline = joblib.load(model_file)
        self.vectorizer = self.pipeline.named_steps["tfidf"]
        self.model = self.pipeline.named_steps["model"]
        self.labels = list(self.model.classes_)          # ['ham', 'spam']
        self.words = self.vectorizer.get_feature_names_out()
        self.weights = word_weights(self.model)
```

`joblib.load` reads the file that training saved. `named_steps["tfidf"]` pulls one step back out of the pipeline by the name it was given in train.py. `classes_` lists the labels in alphabetical order, so index 0 is ham and index 1 is spam. All of this happens **once**, when the server starts, so each prediction is fast.

analyze(text): traced with a real message

Message: `WINNER!! Claim your FREE prize at http://win.biz now`

```
numbers = self.vectorizer.transform([text])
```

Cleans the text to `winner claim your free prize at url now`, then turns it into 200,000 numbers, of which **11 are non-zero**.

```
word_ids = numbers.indices          # positions of the 11 non-zero words
contributions = numbers.data * self.weights[word_ids]
```

`numbers.indices` holds the positions of the 11 words, and `numbers.data` their TF-IDF values. Multiplying each value by that word's learned weight gives its **contribution**. These are your model's real numbers:

Word or pair	TF-IDF value	Learned weight	Contribution
claim	0.263	+4.316	**+1.134**
free	0.186	+6.007	**+1.115**
prize	0.302	+3.181	**+0.960**
your	0.105	+8.801	**+0.927**
url	0.307	+2.346	+0.719
winner	0.334	+1.379	+0.461
now	0.157	+2.775	+0.436
claim your	0.375	+1.089	+0.408
at url	0.545	+0.543	+0.296
your free	0.336	−0.158	−0.053
at	0.117	−2.181	−0.255
starting value (intercept)			−0.934
Total score			**+5.214**

The intercept is the model's starting lean before reading any words; it's slightly negative because most messages are ham.

```
probabilities = self.model.predict_proba(numbers)[0]    # [0.0054, 0.9946]
winner = int(probabilities.argmax())                    # 1, meaning spam
```

Logistic Regression turns the total score into a probability with the **sigmoid** function: 1 / (1 + e^−5.214) = **0.9946**. A score of 0 would give exactly 50%, large positive scores approach 100% spam, and large negative scores approach 100% ham. `argmax()` returns the position of the biggest probability, here 1.

```
direction = 1 if winner == 1 else -1
helped = contributions * direction
order = np.argsort(-helped)[:top]
```

If the winner were ham, the helpful words would be the ones with *negative* contributions, so the sign is flipped. That way "helped the winner" is always a positive number. `np.argsort(-helped)` sorts from biggest to smallest, and `[:top]` keeps the first 4.

The result, which is exactly what the dashboard shows:

```
{"label": "spam", "confidence": 0.9946,
 "top_words": [{"word": "claim", "weight": 1.134}, {"word": "free", "weight": 1.115},
               {"word": "prize", "weight": 0.96}, {"word": "your", "weight": 0.927}]}
```

The same steps on the sentiment model: `I don't like this phone, not good` gives **negative, 99.7% sure**, with top words `not good` (+2.75), `don't like` (+1.95), `not` (+1.26) and `phone not` (+1.10).

Why this explanation is exact, not a guess: for a linear model the score *is* the sum of the contributions plus the intercept, so listing the biggest contributions is a complete account of the decision. Complex models like BERT need approximation tools such as SHAP to explain themselves.

predict_many(texts)

```
return self.pipeline.predict(texts)
```

The fast path for the benchmark: predicts a whole batch at once, without explanations.

8. The two scripts

scripts/download_data.py: fetching the datasets

```
KAGGLE_DATASETS = [
    ("kazanova/sentiment140", "training.1600000.processed.noemoticon.csv", SENTIMENT_FILE),
    ("uciml/sms-spam-collection-dataset", "spam.csv", SMS_SPAM_FILE),
]
```

A list of (Kaggle dataset id, file name inside it, where to save it).

```
def download_from_kaggle(dataset_id, file_name, destination):
    import kagglehub
    folder = Path(kagglehub.dataset_download(dataset_id))
    matches = list(folder.rglob(file_name))
    shutil.copy(matches[0], destination)
```

- `import kagglehub` sits **inside** the function, so if that library is missing, only the Kaggle downloads fail. The Enron download still works.
- `kagglehub.dataset_download` downloads and unzips into a cache folder and returns its path.
- `folder.rglob(file_name)` searches that folder and all its subfolders for the file ("recursive glob").
- `shutil.copy` copies it into `data/`.

`download_enron()` uses `urllib.request.urlretrieve` to fetch the zip from GitHub, extracts the one CSV with `zipfile`, and deletes the zip with `.unlink()`.

`main()` skips any file that already exists, and wraps each Kaggle download in `try/except`, so one failure prints a hint ("download it in your browser, see data/README.md") instead of stopping everything. You didn't need this script: I put your three files in place directly.

scripts/benchmark.py: scoring every record

The dashboard shows one message at a time so you can watch. This script shows the other side of a real system: clearing a large backlog as fast as possible.

```
BATCH_SIZE = 50_000

def score_in_batches(analyzer, texts):
    for start in range(0, len(texts), BATCH_SIZE):
        batch = texts[start:start + BATCH_SIZE]
        analyzer.predict_many(batch)
        print(f"    {scored:,} / {len(texts):,}", end="\r")
```

- `range(0, 1600000, 50000)` gives 0, 50000, 100000, ..., so 32 batches for the tweets.
- `texts[start:start + BATCH_SIZE]` slices out one batch (Day 1 slicing).
- Predicting 50,000 at once is much faster than one at a time, because the work happens inside NumPy's compiled code (Day 2's vectorisation lesson).
- `end="\r"` returns the cursor to the start of the line, so the progress counter overwrites itself instead of printing 32 lines.

Your result: **1,635,663 records in 42.5 seconds**, about 38,500 per second. Tweets ran at about 48,000 per second, but

emails only about 4,000 per second, because emails are much longer and take longer to clean. The total is saved to `models/benchmark.json`, and the dashboard's model card shows it.

9. app/main.py: the web server and the live stream

Started with `python -m uvicorn app.main:app`, which means: "in the file `app/main.py`, find the object called `app` and serve it". Uvicorn is the web server; FastAPI is the framework that decides what each URL does.

The file has four parts: **shared state**, **helper functions**, the **stream engine**, and the **URLs (API)**.

Part 1: shared state

```
analyzers = {}       # "spam" -> TextAnalyzer, "sentiment" -> TextAnalyzer
feeds = {}           # "spam" -> endless supply of test messages
dashboards = set()   # every browser tab currently connected
recent_events = deque(maxlen=40)   # the 40 newest results
stream = {"paused": False, "speed": 20}
totals = {}          # {"spam": {"processed": 0, "flagged": 0, "correct": 0}, ...}
chart_history = {}   # second -> counts, for the last 60 seconds
```

These variables live at the top of the file, so there's **one copy for the whole server**. Every browser and every request sees the same values. That's why pausing in one tab pauses all tabs. In ASP.NET terms, it's like registering services as singletons.

`deque(maxlen=40)` is a list with a size limit: when a 41st item is added, the oldest one drops off the other end automatically.

Part 2: helper functions

`endless_messages(stream_file)`: a generator.

```
def endless_messages(stream_file):
    messages = pd.read_csv(stream_file)
    round_number = 0
    while True:
        shuffled = messages.sample(frac=1, random_state=RANDOM_SEED + round_number)
        for row in shuffled.itertuples(index=False):
            yield row.text, row.label, row.source
        round_number += 1
```

The keyword `yield` makes this a **generator**: a function that hands out one value, pauses right there, and continues from the same spot the next time someone asks with `next(...)`. It's like C#'s `yield return`. Because of `while True`, it never runs out: after the last of the 320,000 tweets it reshuffles (`sample(frac=1)` means "all rows, in random order") and starts again. This is why your feed never stops.

`load_models()` runs once at startup. For each task, if its model file and stream file exist, it creates a `TextAnalyzer`, a generator and zeroed counters. If not, it prints `No sentiment model yet. Run: python -m src.train`, and the dashboard shows "not trained" for that model.

`is_explicit(text)` and `display_text(text)`: safe display mode.

```
EXPLICIT_PATTERN = re.compile(r"\b(?:sex|sexy|...)\b|\b(?:porn|erotic|...)\w*", re.IGNORECASE)
```

The pattern is built by joining the word lists from config.py with `|` (OR). `\b` means "word boundary", so `\b(?:sex)\b` matches "sex" but not "Essex". The `\w*` after the word starts matches the rest of a word, so "porn" also catches "pornography". `display_text()` returns "Message hidden in safe display mode" for explicit messages, and otherwise replaces real `@usernames` with `@user` and cuts the text to 280 characters. **It only changes what is displayed**; the model always reads the original text.

`add_to_chart_history(task, is_flagged)` adds one result to the bucket for the current second and deletes buckets older than 60 seconds. `dict.setdefault(key, default)` means "give me this key's value, creating it with the default if it's missing".

`send_to_all_dashboards(event)` loops over every connected browser and sends the result. If sending fails, that tab has been closed, so it's removed from the set. `list(dashboards)` makes a copy first, because you can't remove items from a set while looping over that same set.

Part 3: the stream engine

```
async def run_live_stream():
    task_order = cycle(list(analyzers))      # spam, sentiment, spam, sentiment, ...
    while True:
        if stream["paused"] or not analyzers:
            await asyncio.sleep(0.2); continue
        task = next(task_order)
        text, true_label, source = next(feeds[task])
        result = analyzers[task].analyze(text)
        is_correct = result["label"] == true_label
        ... update totals and chart history ...
        event = {"task": task, "source": source, "text": display_text(text),
                 "prediction": ..., "confidence": ..., "top_words": ...,
                 "true_label": true_label, "correct": is_correct, "time": time.time()}
        recent_events.append(event)
        await send_to_all_dashboards(event)
        await asyncio.sleep(1 / stream["speed"])
```

This is the heart of “real time”. Each loop takes one message, predicts it, checks it against the true label, records it, and pushes it to every browser. Then it waits: at 20 msg/s, `1 / 20 = 0.05` seconds.

- `cycle([...])` from `itertools` repeats a list forever, so the two models take turns.
- `totals[task]["correct"] += is_correct` works because Python treats `True` as 1 and `False` as 0.
- **`async`** / **`await`** is the same idea as C#’s `async Task`. During `await asyncio.sleep(...)`, the server isn’t frozen: it keeps answering other requests, like your Analyse button, between stream messages. That’s how one Python process does both jobs at once.

Starting and stopping: lifespan

```
@asynccontextmanager
async def lifespan(app):
    load_models()                                  # at startup
    engine = asyncio.create_task(run_live_stream()) # start the loop in the background
    yield                                          # the server runs here
    engine.cancel()                                # at shutdown (Ctrl+C)

app = FastAPI(title="TextPulse", lifespan=lifespan)
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")
```

`asyncio.create_task` starts the stream loop running alongside the server instead of waiting for it to finish (it never would). `app.mount("/static", ...)` serves every file in `app/static` at URLs starting with `/static/`, which is how the browser downloads `style.css` and `app.js`.

Part 4: the URLs

Decorator	Function	What it returns
`@app.get("/")`	`dashboard_page()`	The `index.html` file
`@app.get("/api/status")`	`status()`	Models, pause state, speed, totals, 40 recent results, 60 s of chart history
`@app.get("/api/metrics")`	`metrics()`	`metrics.json` plus `benchmark.json`
`@app.post("/api/analyze")`	`analyze(request)`	Both models’ answers for your text
`@app.post("/api/stream/speed")`	`set_speed(request)`	Changes `stream["speed"]`
`@app.post("/api/stream/pause")` / **`resume`**	`pause()` / `resume()`	Sets `stream["paused"]`
`@app.websocket("/ws/live")`	`live_updates(connection)`	Keeps a connection open for pushed results

A decorator (the `@` line) attaches the function below it to a URL, like `[HttpGet("api/status")]` on an ASP.NET controller method. Returning a Python dict automatically sends JSON.

Request checking with Pydantic:

```
class AnalyzeRequest(BaseModel):
    text: str = Field(min_length=1, max_length=5000)

@app.post("/api/analyze")
def analyze(request: AnalyzeRequest):
    return {task: analyzer.analyze(request.text) for task, analyzer in analyzers.items()}
```

FastAPI reads the JSON body, checks it against `AnalyzeRequest`, and rejects bad input before your function runs. Sending an empty text returns error 422 with `String should have at least 1 character`. It’s the same idea as a DTO with `[Required]` and `[MaxLength]` in ASP.NET.

The WebSocket:

```
@app.websocket("/ws/live")
async def live_updates(connection: WebSocket):
    await connection.accept()
    dashboards.add(connection)
    try:
        while True:
            await connection.receive_text()
    except WebSocketDisconnect:
        dashboards.discard(connection)
```

Accepts the browser's connection and remembers it in `dashboards`, so the stream engine can push results to it. The `while True` loop only waits; its purpose is to notice when the browser disconnects (tab closed or refreshed), so the connection can be forgotten.

10. app/static: the dashboard in your browser

Three files split the work the way every website does: **HTML is the structure** (what's on the page), **CSS is the look** (colours, sizes, layout), and **JavaScript is the behaviour** (what changes and when). All three run in your browser, not in Python.

index.html: the structure

The page is a set of boxes, each with an `id` so JavaScript can find it:

Section	Main ids	Filled in by
Top bar	`status`, `speed`, `pause`	`setStatus()`, the speed and pause handlers
Five tiles	`kpi-total`, `kpi-rate`, `kpi-spam`, `kpi-positive`, `kpi-accuracy`	`drawTotals()`, `drawThroughput()`
Two charts	`<canvas id="spam-chart">`, `<canvas id="sentiment-chart">`	Chart.js, through `updateCharts()`
Live feed	`<tbody id="feed">` (empty at first)	`feedRow()` adds rows
Analyse box	`analyze-form`, `analyze-text`, `analyze-result`	the submit handler
Model card	`model-card`	`loadModelCard()`

Four lines load everything else:

```
<link rel="stylesheet" href="https://fonts.googleapis.com/..."> <!-- fonts (needs internet; falls back if offline) -->
<link rel="stylesheet" href="/static/style.css">                <!-- the look -->
<script src="/static/vendor/chart.umd.js"></script>             <!-- Chart.js, saved in the project -->
<script src="/static/app.js"></script>                         <!-- the behaviour, loaded last -->
```

`app.js` is loaded at the very bottom so every box above already exists when its code starts looking for them. Chart.js is stored inside the project (`static/vendor`) instead of loaded from the internet, so the dashboard works during a presentation even without Wi-Fi.

style.css: the look

Colour variables (tokens). Every colour is defined once as a variable, then used everywhere by name:

```
:root { --spam: #d1344f; --ham: #0f7c86; --positive: #2b8a3e; --negative: #c4580c; ... }
@media (prefers-color-scheme: dark) {
  :root { --spam: #ff7a8f; --ham: #4fc3cc; ... }
}
.badge.spam { background: var(--spam-soft); color: var(--spam); }
```

`@media (prefers-color-scheme: dark)` applies only when Windows is set to dark mode. It swaps the variable values, and every element that uses them changes automatically. That's how the whole dashboard follows your system theme without any JavaScript.

Layout with grid.

```
.kpis { display: grid; grid-template-columns: repeat(auto-fit, minmax(190px, 1fr)); gap: 12px; }
.lower { display: grid; grid-template-columns: minmax(0, 1fr) 380px; }
@media (max-width: 1000px) { .lower { grid-template-columns: 1fr; } }
```

- The tiles row says: "fit as many columns as you can, each at least 190 px wide, sharing the space equally". On a wide screen that's 5 across; on a phone, 1 per row.
- The lower section is the feed (all remaining space) plus a 380 px side column. Below 1,000 px wide it becomes one column.

Small details: `.status.live .dot { animation: pulse ... }` makes the green dot pulse; `.feed tr.fresh` briefly highlights new rows; `.confidence-text.low` turns the "· low" label orange; `[hidden] { display: none !important; }` makes sure anything

JavaScript marks as hidden really disappears.

app.js: the behaviour

Start-up: `start()` runs once when the page loads.

```
async function start() {
  const status = await (await fetch("/api/status")).json();
  loadedModels = status.models;                          // ["spam", "sentiment"]
  for (const task of loadedModels) Object.assign(totals[task], status.totals[task]);
  waitingEvents = status.recent;                         // 40 newest results for the feed
  for (const bucket of status.chart_history) perSecond.set(...);   // last 60 s for the charts
  updateCharts(); drawThroughput(); drawTotals();
  loadModelCard();
  connect();                                             // open the WebSocket
  requestAnimationFrame(drawWaitingEvents);              // start the drawing loop
  setInterval(updateCharts, 1000);                       // redraw charts every second
  setInterval(drawThroughput, 1000);
}
```

`fetch(url)` is the browser's way to call an API, like `HttpClient.GetAsync` in C#. Loading the totals, recent results and chart history from the server is why a refresh shows everything immediately.

Receiving results: `connect()`.

```
const socket = new WebSocket(`ws://${location.host}/ws/live`);
socket.onmessage = (message) => {
  const event = JSON.parse(message.data);   // text -> JavaScript object
  countEvent(event);                        // update totals and this second's chart bucket
  waitingEvents.push(event);                // queue it for drawing
};
socket.onclose = () => { setStatus("offline", "Offline, retrying"); setTimeout(connect, 2000); };
```

`onmessage` runs every time the server pushes a result. `onclose` runs if the server stops; after 2 seconds it calls `connect()` again, so restarting the server reconnects the dashboard by itself.

Counting: `countEvent(event)` adds 1 to processed, adds 1 to correct if the server marked it ✓, adds 1 to flagged if the prediction is "spam" or "positive", and adds the result to the bucket for the current second (`Math.floor(Date.now() / 1000)`).

Drawing efficiently: `drawWaitingEvents()`.

```
function drawWaitingEvents() {
  if (waitingEvents.length) {
    for (const event of waitingEvents.slice(-60)) feed.prepend(feedRow(event));
    while (feed.rows.length > 60) feed.deleteRow(-1);
    waitingEvents = [];
    drawTotals();
  }
  requestAnimationFrame(drawWaitingEvents);
}
```

At 200 msg/s, redrawing the page for each message would make the browser stutter. So results wait in a queue, and `requestAnimationFrame` calls this function once per screen refresh (about 60 times a second) to draw everything waiting at once. `prepend` puts new rows at the top, and `deleteRow(-1)` removes the oldest from the bottom so there are never more than 60.

Building one row: `feedRow(event)` creates a `<tr>` with the time, source, message, word tags, prediction badge, confidence bar (`width: 88%` for 88% sure) and ✓/✗. Every piece of message text goes through `escapeHtml()` first:

```
function escapeHtml(text) {
  const div = document.createElement("div");
  div.textContent = text;     // stored as plain text, never run as code
  return div.innerHTML;       // "<script>" comes back as "&lt;script&gt;"
}
```

The messages come from real spammers. If one contained `<script>`, inserting it directly into the page would run that code in your browser. This is called **XSS (cross-site scripting)**, and escaping prevents it.

Charts: `updateCharts()` runs every second. For each of the last 60 *completed* seconds it calculates `100 × spam / spamTotal` and `100 × positive / sentimentTotal` from the buckets, then gives the numbers to Chart.js. Seconds with no data become `null`, so there's no line there.

Throughput: `drawThroughput()` adds up all results in the last 5 completed seconds and divides by 5.

Buttons:

- **Pause/Resume** sends `POST /api/stream/pause` or `/resume`, then changes the button text and status badge.
- **Speed** sends `POST /api/stream/speed` with `{"messages_per_second": 50}`.
- **Tabs** (All / Spam model / Sentiment model) only set `row.hidden` on rows in your browser. Nothing goes to the server.

- **Analyse** sends your text to `POST /api/analyze` and shows both models' answers. The sample chips fill the text box and submit the form for you.

Model card: `loadModelCard()` fetches `/api/metrics` once and builds the score boxes and comparison tables from `metrics.json` and `benchmark.json`.

11. Tests, models and docs

tests/test_textpulse.py: automatic checks

Run with `python -m pytest`. pytest finds every function whose name starts with `test_`, runs it, and reports any `assert` that fails. `assert x == y` means "stop with an error if x isn't y".

```
def test_clean_text_replaces_links_and_usernames():
    assert clean_text("Visit http://win.com @john") == "visit url user"
```

The four `clean_text` tests pin down the cleaning rules: lowercase and punctuation, links and usernames, keeping "don't" and "not", and numbers-only input. If someone later edits the regex and breaks a rule, the test fails immediately.

```
TINY_TEXTS = ["win a free prize now", "free cash click here", "claim your free reward",
              "team meeting on friday", "lunch with the team", "project review friday"]

@pytest.mark.parametrize("algorithm", [MultinomialNB(), LogisticRegression()])
def test_analyzer_predicts_and_explains(tmp_path, algorithm):
    analyzer = TextAnalyzer(save_tiny_model(tmp_path, algorithm))
    spam = analyzer.analyze("free prize, click now")
    assert spam["label"] == "spam"
    assert "free" in [w["word"] for w in spam["top_words"]]
```

- The analyzer tests train a **tiny model on 6 messages**, so they run in about a second without the 290 MB of real data.
- `@pytest.mark.parametrize` runs the same test twice, once per algorithm, because `word_weights()` has a different branch for each.
- `tmp_path` is a temporary folder that pytest creates and deletes, so tests never touch your real `models/` folder.

That's 4 + 2 + 1 = **7 tests**, which is the "7 passed" you saw.

models/: what training creates

File	Size on your laptop	Created by	Read by
`spam_model.joblib`	3.9 MB	train.py	analyzer.py (server, benchmark)
`sentiment_model.joblib`	3.3 MB	train.py	analyzer.py
`stream_spam.csv`	4.2 MB, 7,133 messages	train.py	main.py's stream
`stream_sentiment.csv`	29 MB, 320,000 tweets	train.py	main.py's stream
`metrics.json`	4 KB	train.py	`/api/metrics`, the model card
`benchmark.json`	0.2 KB	benchmark.py	`/api/metrics`, the model card

The models are only 3–4 MB even though they learned from 1.6 million tweets. They store just the vocabulary and one weight per word, not the tweets themselves.

docs/

- `HOW_IT_WORKS.md`: a shorter version of this walkthrough that lives with the code on GitHub.
- `RUN_IN_VSCODE.md`: the setup steps you followed, with fixes for common errors.
- `dashboard.png`: the screenshot at the top of the README.

12. One message from start to finish

Follow the spam email `WINNER!! Claim your FREE prize at http://win.biz now` through every file it touches.

Earlier, during training (once):

1. **load_data.py** reads it from the Enron CSV as one row: text, label `spam`, source `Email`.
2. **train.py** puts it in either the 80% training set or the 20% test set. Say it lands in the test set: the model never learns from it.
3. **train.py** saves it into `stream_spam.csv`, and saves the trained spam model as `spam_model.joblib`.

When you run `uvicorn app.main:app`:

4. **main.py** `load_models()` creates a `TextAnalyzer` (**analyzer.py**), which loads `spam_model.joblib` and its 200,000 word weights.

5. **main.py** `endless_messages()` reads `stream_spam.csv` and shuffles it.

The moment it streams (about 1 ms of work):

6. **main.py** `run_live_stream()`: it's the spam model's turn, so `next(feeds["spam"])` yields our message with its true label `spam`.
7. **analyzer.py** `analyze()`: the vectorizer calls **clean_text.py**, giving `winner claim your free prize at url now`, then turns it into 11 non-zero TF-IDF numbers.
8. Contributions are added up: claim +1.13, free +1.12, prize +0.96, your +0.93, … intercept −0.93, total **+5.21**. The sigmoid turns that into **99.46% spam**.
9. **main.py** checks `"spam" == "spam"`, so `correct` is True. It adds 1 to processed, flagged and correct, and adds the result to this second's chart bucket.
10. **main.py** `display_text()` finds no explicit words, so the text is shown unchanged.
11. **main.py** builds the event and `send_to_all_dashboards()` sends it as JSON over the WebSocket:

```
{"task": "spam", "source": "Email", "text": "WINNER!! Claim your FREE prize at http://win.biz now",
 "prediction": "spam", "confidence": 0.9946,
 "top_words": [{"word": "claim", "weight": 1.134}, {"word": "free", "weight": 1.115}, ...],
 "true_label": "spam", "correct": true, "time": 1793461075.2}
```

In your browser (app.js):

12. `socket.onmessage` receives it, and `countEvent()` updates the totals and the chart bucket.
13. On the next screen refresh, `drawWaitingEvents()` calls `feedRow()`, which escapes the text and builds the row: a red **Spam** badge, a 99% bar, the tags `claim free prize your`, and a green ✓. **style.css** supplies the red, the bar and the highlight animation.
14. Within a second, `updateCharts()` includes it in the Spam rate line, and the **Spam detected** and **Live accuracy** tiles move slightly.

0.05 seconds later, `asyncio.sleep(1 / 20)` ends, and step 6 repeats with a tweet for the sentiment model.

13. Check yourself

Try to answer each one without looking back, then check the section in brackets. If you can answer all of them, you can explain the project to a panel.

1. Why does every path in the project start from `PROJECT_DIR` instead of a typed path like `D:\AI ML\...`? *(3)*
2. What does `clean_text` turn `Visit www.deals.com @priya NOW!!!` into, and why are links kept as a word instead of deleted? *(4)*
3. Why were 3,625 duplicate messages removed, and what would have happened to the scores if they hadn't been? *(5)*
4. Why is the vocabulary learned from 300,000 tweets instead of all 1.28 million, and why only from training data? *(6)*
5. In TF-IDF, why does `at` get a much smaller value than `at url`? *(6)*
6. From the spam confusion matrix, which mistake is worse for a real email user: the 58 or the 88? Which score, precision or recall, measures it? *(6)*
7. How does the analyzer know `claim` was the most important word in the example? Why can we call that explanation exact? *(7)*
8. A total score of 0 gives what probability? What about a large negative score? *(7)*
9. Why does the live feed never end? Which Python keyword makes this possible? *(9)*
10. How can the server stream messages and answer your Analyse button at the same time? *(9)*
11. Why does pausing in one browser tab pause every tab? *(9)*
12. Why does the dashboard queue results and draw them with `requestAnimationFrame`, instead of drawing each one immediately? *(10)*
13. What could go wrong without `escapeHtml()`? *(10)*
14. Why do the tests train a 6-message model instead of loading the real one? *(11)*
15. Why are the trained models only about 3 MB when they learned from 1.6 million tweets? *(11)*

Try changing things (restart the server or retrain afterwards, and see what happens):

- In `src/train.py`, change `ngram_range=(1, 2)` to `(1, 1)` and retrain. How much does sentiment accuracy drop without word pairs like "not good"?
- In `src/config.py`, set `STREAM_START_SPEED = 2` to watch the stream in slow motion.
- In `app/static/style.css`, change `--spam` to a different colour and refresh the page.
- In the dashboard, analyse `free lunch at the team meeting`. Which words push toward spam, and which push back?

Send me your answers in chat and I'll check them.